Filière : Cyber Sécurité et Intelligence Artificielle (CSIA)

Module : Architecture Logicielle

Rapprot

Nom et Prénom : Abderrazak MOSTAID

Objectif du TP

- Créer une application Spring Boot monolithique simple
- Développer une API REST CRUD complète
- Comprendre l'architecture en couches (Repository, Service, Controller)
- Découvrir RestRepository pour simplifier le code
- Identifier les avantages et limites d'un monolithe

Compétences acquises

A la fin de ce TP, il faut découvrir l'outil Spring-Boot pour développer des applications basées sur le langage du programmation JAVA, aussi l'utilisation du **Postaman** comme outil pour le teste des requêtes http, entre l'interface est les Endpoints crée.

Aussi découvrir d'une manière profonde l'architecture monolithique simple, est comment utiliser RestRepository pour simplifier le code.

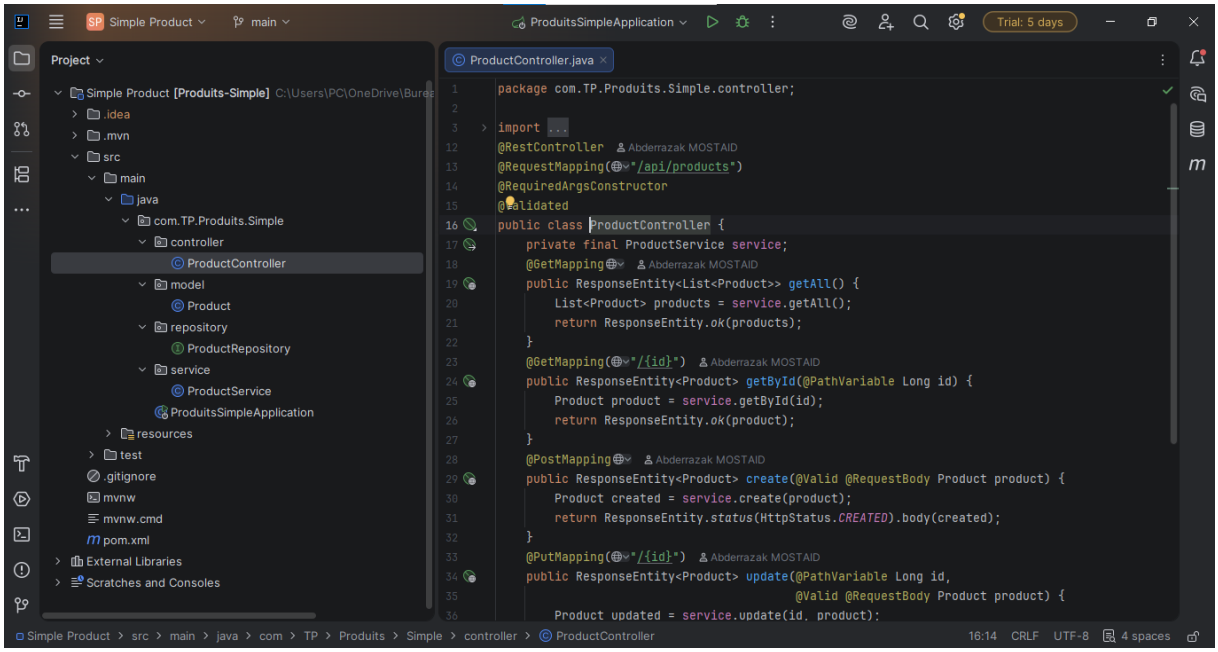
De plus on va découvrir les avantages et les limites de cette architecture.

La partie pratique

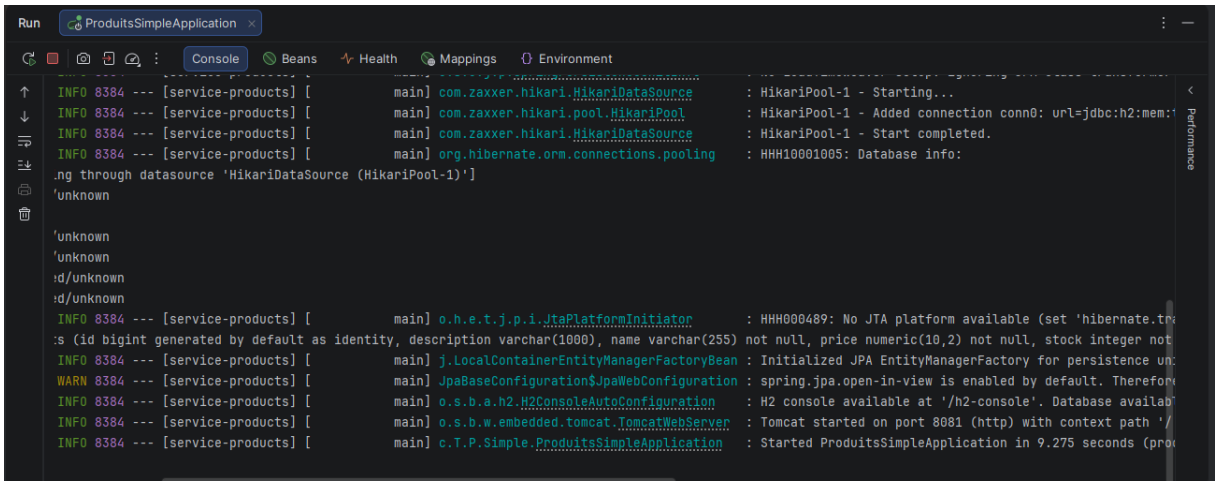
Pour cette partie, le TP généralement est diviser en deux grandes parties. La première consiste à développer le code manuellement avec la création des packages et les modules nécessaires (Repository, controller, service), et la deuxième consiste à utiliser le RestRepository pour simplifier le code et éviter l'utilisation du controller.

Partie 1

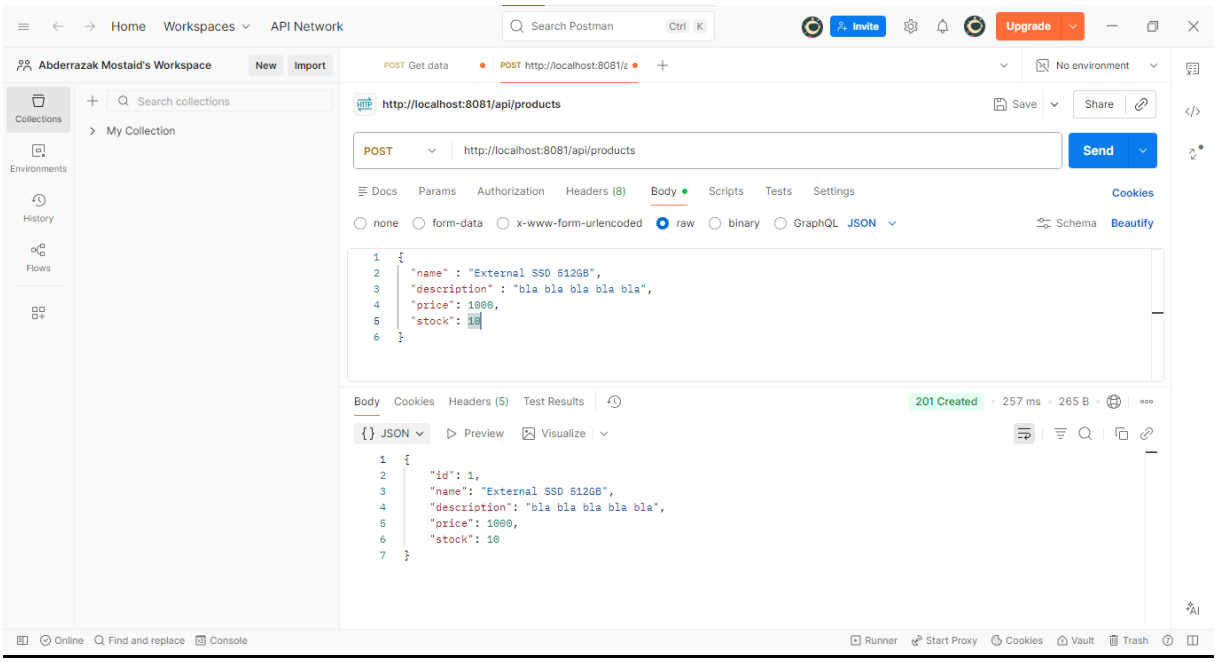
Structure Global du Projet



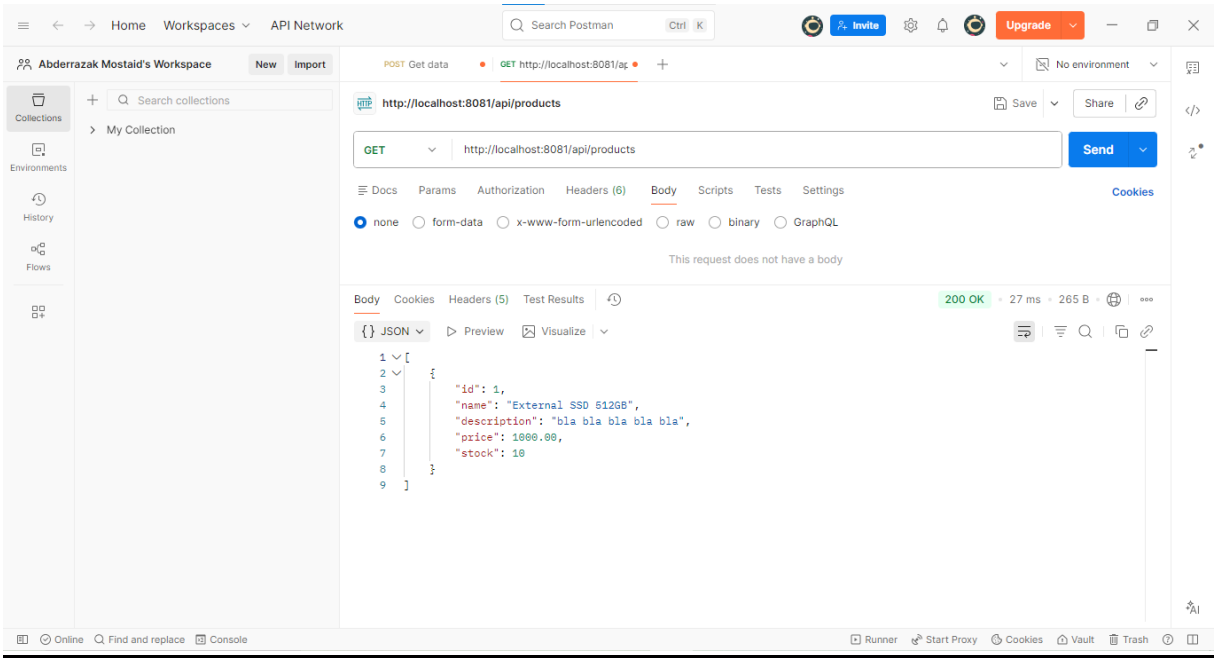
Exécution du code



Vérification avec Postman (Post request)



Vérification avec Postman (Get request)

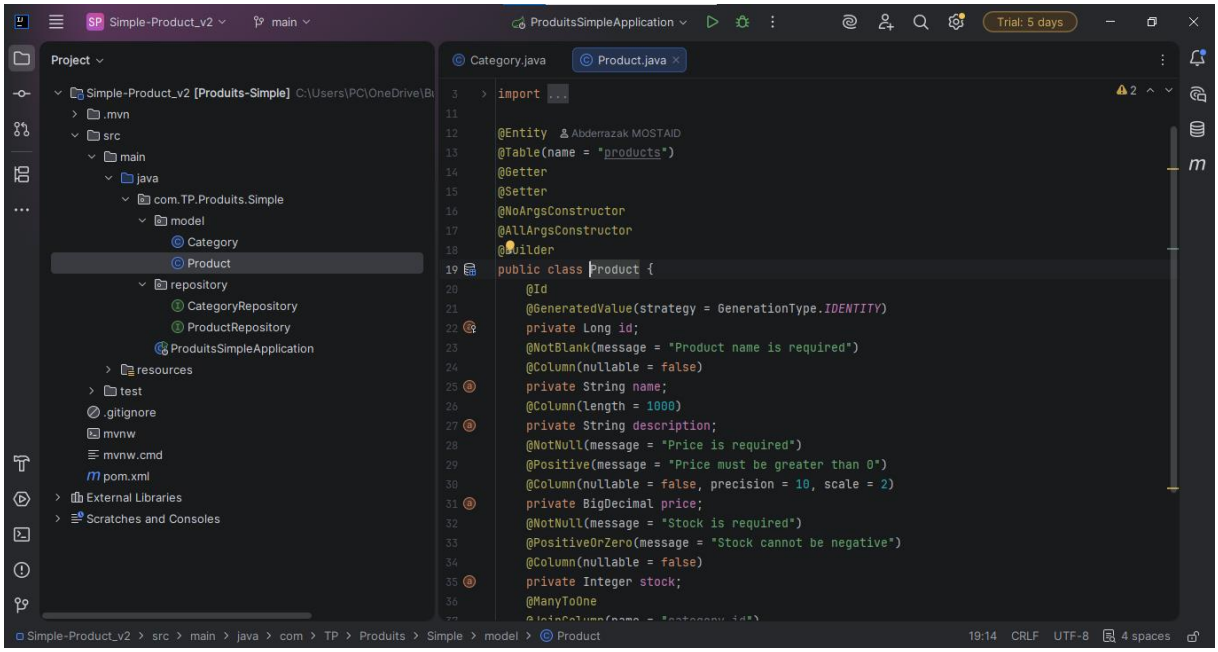


Ici on a vérifié que la partie 1 a été traité avec succès, on va donc passer à la partie 2 ou on va découvrir RestRepository.

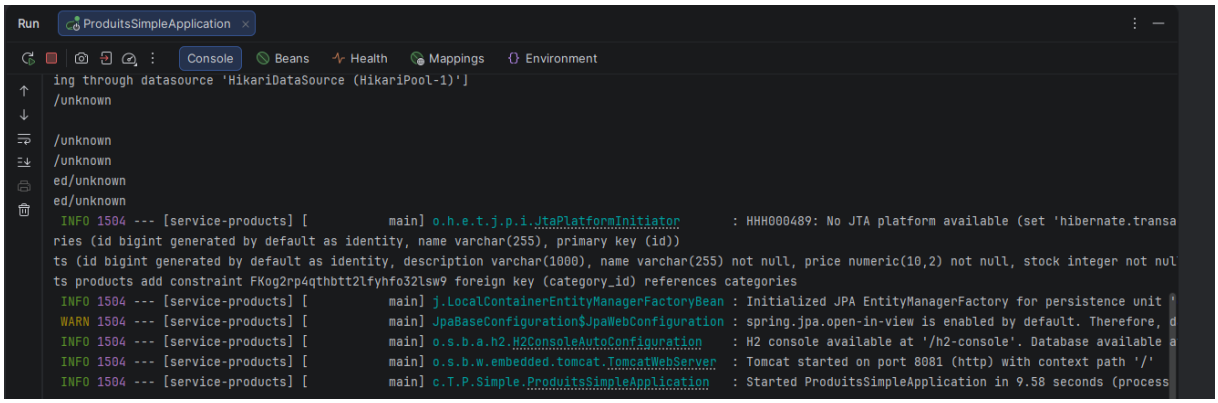
Partie 2

RestRepository est un concept utilisé avec **Spring Data REST** pour **exposer automatiquement un repository JPA comme une API REST**, sans écrire de contrôleur.

Structure Global du Projet



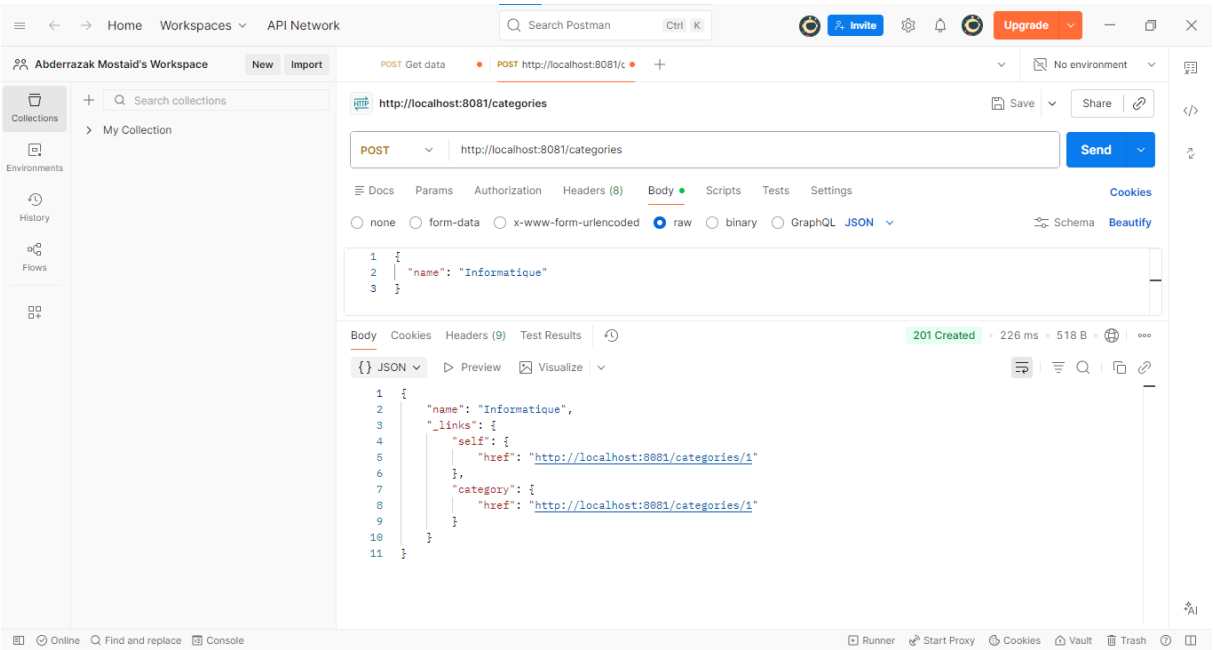
Exécution du code



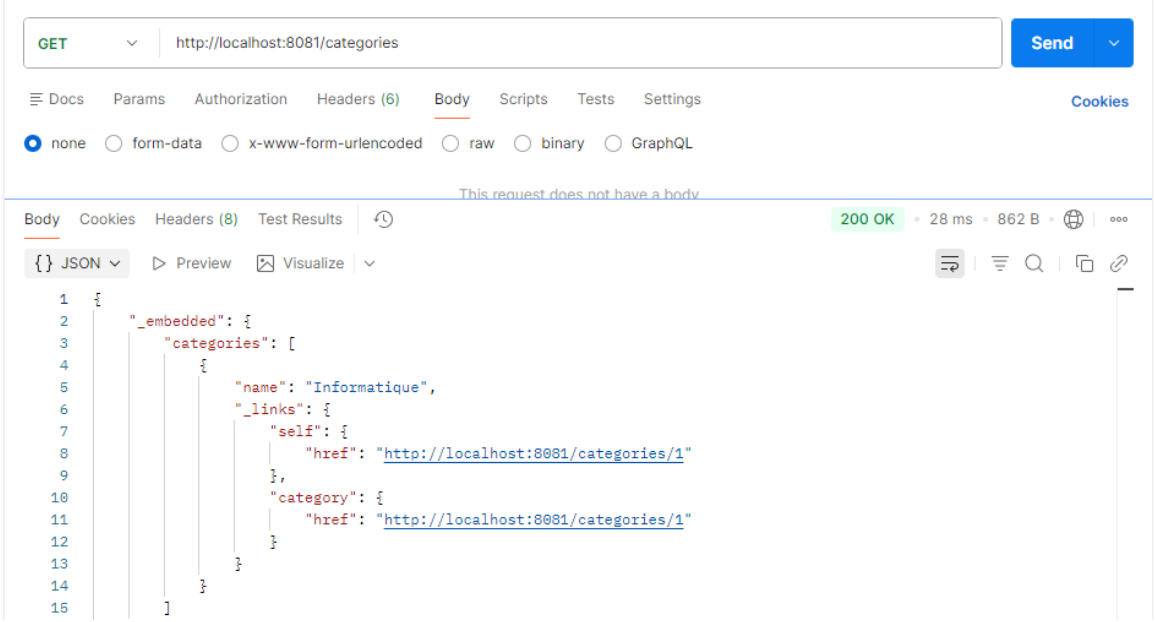
Ce qui observer comme déférence est que le contrôleur n'est pas utilisé lors de l'utilisation du RestRepository.

Maintenant avec la vérification avec les requêtes http dans Postman pour vérifier si tous est marche bien.

Vérification avec Postman (Post)



Vérification avec Postman (Get)



Pour accélérer on va créer une teste sur un produit qui appartient à la catégorie que nous avons déjà créée.

Post request

POST http://localhost:8081/products Send

Docs Params Authorization Headers (8) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1  {
2    "name": "Laptop",
3    "description": "Lenovo Thinkpad",
4    "price": 9000,
5    "stock": 5,
6    "category": "http://localhost:8081/categories/1"
7  }
```

Body Cookies Headers (9) Test Results 201 Created 94 ms 666 B

{} JSON Preview Visualize

```
1  {
2      "name": "Laptop",
3      "description": "Lenovo Thinkpad",
4      "price": 9000,
5      "stock": 5,
6      "_links": {
7          "self": {
8              "href": "http://localhost:8081/products/1"
9          },
10         "product": {
11             "href": "http://localhost:8081/products/1"
12         },
```

Get Request

Body Cookies Headers (8) Test Results 200 OK 26 ms 1.01 KB

{} JSON Preview Visualize

```
2      "_embedded": {
3          "products": [
4              {
5                  "name": "Laptop",
6                  "description": "Lenovo Thinkpad",
7                  "price": 9000.00,
8                  "stock": 5,
9                  "_links": {
10                     "self": {
11                         "href": "http://localhost:8081/products/1"
12                     },
13                     "product": {
14                         "href": "http://localhost:8081/products/1"
15                     },
16                     "category": {
17                         "href": "http://localhost:8081/products/1/category"
18                     }
19                 }
```

Runner Start Proxy Cookies Vault Trash

Finalement on va vérifier la relation

GET http://localhost:8081/products/1/category Send

Docs Params Authorization Headers (6) Body Scripts Tests Settings Cookies

Body Cookies Headers (9) Test Results 200 OK 21 ms 521 B

Conclusion

L’architecture monolithique consiste à développer une application comme un bloc unique, où toutes les fonctionnalités (interface utilisateur, logique métier et accès aux données) sont regroupées dans une seule application. Cette approche présente plusieurs avantages, notamment sa simplicité de conception et de déploiement, ce qui la rend adaptée aux petits projets ou aux applications au début de leur développement. Elle facilite également les tests et le suivi de l’application, car tout le code est centralisé. Cependant, cette architecture présente certaines limites lorsque l’application devient plus complexe : elle peut devenir difficile à maintenir et à faire évoluer, car toute modification nécessite souvent de redéployer l’ensemble du système. De plus, la scalabilité est limitée, puisque l’application doit être mise à l’échelle dans sa totalité plutôt que par composants indépendants. Ainsi, bien que l’architecture monolithique soit efficace pour des projets simples ou de taille moyenne, elle peut devenir contraignante pour des systèmes de grande envergure nécessitant plus de flexibilité et de modularité.